



6 LUGLIO 2026

**Report di validazione AVALLA, ATGT
e AsmetaSMV del modello ASM
dell'ascensore**

MAFFEIS RICCARDO 1085706



Sommario

Validazione tramite scenari AVALLA	2
Scenari considerati.....	2
Esito complessivo degli scenari AVALLA	2
Dettaglio degli scenari validati.....	3
1. Idle ascensore.....	3
2. Richiesta verso piano superiore.....	5
3. Richiesta verso piano inferiore	7
4. Overload	9
5. Risoluzione overload.....	11
6. Richieste durante overload.....	13
7. Guasto tecnico	15
8. Blocco delle richieste durante guasto	17
9. Conservazione delle richieste durante guasto.....	19
10. Ripristino dopo guasto.....	21
Generazione automatica di scenari con ATGT.....	23
Model checking con AsmetaSMV	24
Conclusione	27

Validazione tramite scenari AVALLA

Questo documento presenta i risultati della validazione del modello ASM relativo al sistema di ascensore. La validazione è stata eseguita mediante scenari AVALLA, ciascuno dedicato a una specifica situazione operativa del sistema. Gli scenari sono stati validati anche con analisi di coverage, utilizzata come supporto per valutare la copertura delle principali regole del modello.

Scenari considerati

La validazione è stata suddivisa in più scenari, così da verificare separatamente le principali funzionalità del modello. Gli scenari considerati sono i seguenti:

- funzionamento in assenza di richieste;
- richiesta verso un piano superiore;
- richiesta verso un piano inferiore;
- gestione del sovraccarico;
- risoluzione del sovraccarico;
- acquisizione di richieste durante il sovraccarico;
- ingresso nello stato di guasto;
- blocco delle nuove richieste durante il guasto;
- conservazione delle richieste già acquisite durante il guasto;
- ripristino dopo guasto.

Ogni scenario è contenuto in un file .avalla separato, in modo che la simulazione parta sempre dallo stato iniziale definito nel modello ASM.

Esito complessivo degli scenari AVALLA

Tutti gli scenari validati terminano senza errori.

Nel complesso, la validazione conferma che il modello rispetta i principali requisiti attesi:

- la cabina non si muove con porte aperte;
- le richieste vengono acquisite e mantenute finché non sono servite;
- il movimento avviene in modo coerente con la direzione scelta;
- lo stato di OVERLOAD blocca la cabina e mantiene le porte aperte;
- lo stato di GUASTO blocca la cabina e sospende l'acquisizione di nuove richieste;
- le richieste già acquisite non vengono eliminate durante le anomalie;
- il sistema può tornare operativo dopo il timeout di ripristino.

Dettaglio degli scenari validati

1. Idle ascensore

```
scenario IdleAscensore
load ascensore.asm

begin nessunaRichiesta
  set eventoGuasto := false;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := false;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := false;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step

  check pianoCorrente = 0;
  check statoCabina = FERMA;
  check statoPorte = CHIUSE;
  check direzione = NESSUNA;
  check statoErrore = NESSUNO;
end
```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciAscensore()</i>	<i>Covered</i>	50.00%	50.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	25.00%	25.00%	0.00%	-
<i>r_idle()</i>	<i>Covered</i>	-	100.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	12.50%	0.00%	-
<i>r_main()</i>	<i>Covered</i>	50.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	50.00%	50.00%	0.00%	33.33%
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-

Descrizione dello scenario

Lo scenario imposta a false tutte le richieste interne ed esterne e disattiva gli eventi di guasto, simulando una condizione di assenza totale di input.

Obiettivo

Verificare che, in assenza di richieste e di condizioni di errore, l'ascensore rimanga fermo nello stato di attesa.

Risultato

La simulazione conferma che la cabina resta al piano iniziale 0, con stato FERMA, porte CHIUSE, direzione NESSUNA e nessun errore attivo.

2. Richiesta verso piano superiore

```

scenario RichiestaPianoSuperiore
load ascensore.asm

begin statoIniziale
  check pianoCorrente = 0;
  check statoCabina = FERMA;
  check statoPorte = CHIUSE;
  check direzione = NESSUNA;
  check statoErrore = NESSUNO;
  check richiesteAttive(3) = false;
end

begin richiestaInternaPiano3
  set eventoGuasto := false;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := false;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := true;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

step

  check richiesteAttive(3) = true;
  check direzione = SU;
  check pianoCorrente = 1;
  check statoCabina = IN_MOVIMENTO;
  check statoPorte = CHIUSE;
end

begin movimentoVersoPiano3
  set pulsanteInterno(3) := false;

  step

  check pianoCorrente = 2;
  check direzione = SU;
  check statoPorte = CHIUSE;

  step

  check pianoCorrente = 3;
  check direzione = SU;
  check statoPorte = CHIUSE;
end

begin servizioPiano3
  step

  check pianoCorrente = 3;
  check richiesteAttive(3) = false;
  check statoCabina = FERMA;
  check statoPorte = APERTE;
  check direzione = NESSUNA;
end

begin chiusuraPorte
  step

  check pianoCorrente = 3;
  check statoCabina = FERMA;
  check statoPorte = CHIUSE;
  check direzione = NESSUNA;
end

```

Regola	Stato	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore()	Covered	75.00%	90.00%	-	-
r_aggiornaPersone()	Covered	75.00%	75.00%	0.00%	-
r_chiudiPorte()	Covered	50.00%	100.00%	100.00%	-
r_gestisciGuasto()	Covered	33.33%	12.50%	0.00%	-
r_scegliDirezione()	Covered	33.33%	36.84%	10.00%	-
r_main()	Covered	50.00%	100.00%	-	-
r_acquisisciRichieste()	Covered	100.00%	100.00%	100.00%	66.67%
r_muoviAscensore()	Covered	33.33%	41.67%	33.33%	-
r_gestisciErrori()	Covered	50.00%	66.67%	-	-
r_serviPianoCorrente()	Covered	50.00%	100.00%	100.00%	-

Descrizione dello scenario

Lo scenario parte con l'ascensore fermo al piano 0, con porte chiuse e nessun errore attivo. Viene poi simulata una richiesta interna verso il piano 3, verificando passo dopo passo il movimento della cabina fino al piano richiesto.

Obiettivo

Verificare che l'ascensore acquisisca e serva una richiesta interna verso un piano superiore.

Risultato

La richiesta verso il piano 3 viene acquisita correttamente. Il modello imposta la direzione a SU e muove progressivamente la cabina dal piano 0 al piano 3. Al raggiungimento del piano richiesto, la cabina si ferma, apre le porte, rimuove la richiesta attiva e annulla la direzione.

3. Richiesta verso piano inferiore

```

scenario RichiestaPianoInferiore
load ascensore.asm

begin portaAscensoreAlPiano3
  set eventoGuasto := false;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := false;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := true;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step
  set pulsanteInterno(3) := false;
  step
  step
  step
  step

  check pianoCorrente = 3;
  check statoPorte = CHIUSE;
end

begin richiestaPiano0
  set chiamataPianoGiu(0) := true;

  step

  check richiesteAttive(0) = true;
  check direzione = GIU;
  check pianoCorrente = 2;
  check statoCabina = IN_MOVIMENTO;
end

begin movimentoVersoPiano0
  set chiamataPianoGiu(0) := false;

  step

  check pianoCorrente = 1;
  check direzione = GIU;

  step

  check pianoCorrente = 0;
  check direzione = GIU;
end

begin servizioPiano0
  step

  check pianoCorrente = 0;
  check richiesteAttive(0) = false;
  check statoCabina = FERMA;
  check statoPorte = APERTE;
  check direzione = NESSUNA;
end

```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciAscensore()</i>	<i>Covered</i>	75.00%	90.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	75.00%	75.00%	0.00%	-
<i>r_chiudiPorte()</i>	<i>Covered</i>	50.00%	100.00%	100.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	12.50%	0.00%	-
<i>r_scegliDirezione()</i>	<i>Covered</i>	55.56%	57.89%	20.00%	-
<i>r_main()</i>	<i>Covered</i>	50.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	100.00%	100.00%	100.00%	66.67%
<i>r_muoviAscensore()</i>	<i>Covered</i>	66.67%	75.00%	66.67%	-
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-
<i>r_serviPianoCorrente()</i>	<i>Covered</i>	50.00%	100.00%	100.00%	-

Descrizione dello scenario

Lo scenario porta inizialmente l'ascensore al piano 3, con porte chiuse e nessun errore attivo. Successivamente viene simulata una chiamata esterna verso il basso dal piano 0, verificando il movimento della cabina fino al piano richiesto.

Obiettivo

Verificare che l'ascensore acquisisca e serva correttamente una richiesta posta sotto il piano corrente.

Risultato

La cabina viene inizialmente portata al piano 3. Successivamente viene acquisita una chiamata esterna verso il basso dal piano 0. Il modello imposta la direzione a GIU e muove progressivamente la cabina fino al piano richiesto. Una volta raggiunto il piano 0, la cabina si ferma, apre le porte, rimuove la richiesta attiva e annulla la direzione.

4. Overload

```

scenario OverloadAscensore
load ascensore.asm

begin apriPorteAlPianoCorrente
    set eventoGuasto := false;
    set personeEntrate := 0;
    set personeUscite := 0;

    set pulsanteInterno(-1) := false;
    set pulsanteInterno(0) := true;
    set pulsanteInterno(1) := false;
    set pulsanteInterno(2) := false;
    set pulsanteInterno(3) := false;
    set pulsanteInterno(4) := false;

    set chiamataPianoSu(-1) := false;
    set chiamataPianoSu(0) := false;
    set chiamataPianoSu(1) := false;
    set chiamataPianoSu(2) := false;
    set chiamataPianoSu(3) := false;
    set chiamataPianoSu(4) := false;

    set chiamataPianoGiu(-1) := false;
    set chiamataPianoGiu(0) := false;
    set chiamataPianoGiu(1) := false;
    set chiamataPianoGiu(2) := false;
    set chiamataPianoGiu(3) := false;
    set chiamataPianoGiu(4) := false;

    step

    check pianoCorrente = 0;
    check statoPorte = APERTE;
    check statoCabina = FERMA;
    check richiesteAttive(0) = false;
end

```

```

begin generaOverload
    set pulsanteInterno(0) := false;
    set personeEntrate := 9;
    set personeUscite := 0;

    step

    check numeroPersone = 9;
    check statoErrore = OVERLOAD;
    check statoCabina = BLOCCATA;
    check statoPorte = APERTE;
    check direzione = NESSUNA;
end

```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciOverload()</i>	<i>Covered</i>	25.00%	50.00%	25.00%	-
<i>r_gestisciAscensore()</i>	<i>Covered</i>	37.50%	40.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	75.00%	75.00%	50.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	12.50%	0.00%	-
<i>r_main()</i>	<i>Covered</i>	75.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	100.00%	100.00%	100.00%	66.67%
<i>r_gestisciErrori()</i>	<i>Covered</i>	100.00%	100.00%	-	-
<i>r_serviPianoCorrente()</i>	<i>Covered</i>	50.00%	100.00%	50.00%	-

Descrizione dello scenario

Lo scenario apre inizialmente le porte dell'ascensore al piano 0. Successivamente viene simulato l'ingresso di 9 persone in cabina, superando la capacità massima consentita pari a 8.

Obiettivo

Verificare che il sistema entri correttamente nello stato OVERLOAD quando il numero di persone presenti in cabina supera la capacità massima consentita.

Risultato

Simulando l'ingresso di 9 persone, a fronte di una capacità massima pari a 8, il modello rileva correttamente il sovraccarico. Il sistema passa nello stato OVERLOAD, blocca la cabina, mantiene le porte aperte e imposta la direzione a NESSUNA.

5. Risoluzione overload

```

scenario RisoluzioneOverload
load ascensore.asm

begin generaOverload
  set eventoGuasto := false;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := true;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := false;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step

  set pulsanteInterno(0) := false;
  set personeEntrate := 9;
  set personeUscite := 0;

  step

```

```

  check numeroPersone = 9;
  check statoErrore = OVERLOAD;
  check statoCabina = BLOCCATA;
  check statoPorte = APERTE;

end

begin rimuoviOverload
  set personeEntrate := 0;
  set personeUscite := 1;

  step

  check numeroPersone = 8;
  check statoErrore = NESSUNO;
  check statoCabina = FERMA;
  check statoPorte = CHIUSE;
  check direzione = NESSUNA;

end

```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciOverload()</i>	<i>Covered</i>	75.00%	100.00%	62.50%	-
<i>r_gestisciAscensore()</i>	<i>Covered</i>	62.50%	60.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	75.00%	75.00%	50.00%	-
<i>r_idle()</i>	<i>Covered</i>	-	100.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	12.50%	0.00%	-
<i>r_main()</i>	<i>Covered</i>	75.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	100.00%	100.00%	100.00%	66.67%
<i>r_gestisciErrori()</i>	<i>Covered</i>	100.00%	100.00%	-	-
<i>r_serviPianoCorrente()</i>	<i>Covered</i>	50.00%	100.00%	50.00%	-

Descrizione dello scenario

Lo scenario genera inizialmente una condizione di sovraccarico portando il numero di persone in cabina a 9. Successivamente viene simulata l'uscita di una persona, riportando il carico entro la capacità massima consentita.

Obiettivo

Verificare che il sistema esca correttamente dallo stato OVERLOAD quando il numero di persone torna entro la capacità massima consentita.

Risultato

Dopo aver generato una condizione di sovraccarico con 9 persone, viene simulata l'uscita di una persona dalla cabina. Il numero di persone passa quindi a 8, rientrando nella capacità massima. Il sistema elimina lo stato di errore, riporta la cabina nello stato FERMA, chiude le porte e mantiene la direzione a NESSUNA.

6. Richieste durante overload

```
scenario RichiesteDuranteOverload
load ascensore.asm

begin generaOverload
  set eventoGuasto := false;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := true;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := false;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step

  set pulsanteInterno(0) := false;
  set personeEntrate := 9;
  set personeUscite := 0;

  step

  check statoErrore = OVERLOAD;
  check statoPorte = APERTE;
end
```

```
begin acquisisciRichiestaDuranteOverload
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(2) := true;

  step

  check statoErrore = OVERLOAD;
  check statoCabina = BLOCCATA;
  check statoPorte = APERTE;
  check richiesteAttive(2) = true;
end
```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciOverload()</i>	<i>Covered</i>	25.00%	50.00%	25.00%	-
<i>r_gestisciAscensore()</i>	<i>Covered</i>	37.50%	40.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	75.00%	75.00%	50.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	12.50%	0.00%	-
<i>r_main()</i>	<i>Covered</i>	75.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	100.00%	100.00%	100.00%	66.67%
<i>r_gestisciErrori()</i>	<i>Covered</i>	100.00%	100.00%	-	-
<i>r_serviPianoCorrente()</i>	<i>Covered</i>	50.00%	100.00%	50.00%	-

Descrizione dello scenario

Lo scenario genera inizialmente una condizione di sovraccarico, portando il sistema nello stato OVERLOAD. Durante questa condizione viene poi simulata una nuova richiesta interna verso il piano 2.

Obiettivo

Verificare che, durante lo stato OVERLOAD, il sistema continui ad acquisire nuove richieste.

Risultato

Durante il sovraccarico viene premuto il pulsante interno del piano 2. Il modello mantiene la cabina bloccata e le porte aperte, ma registra comunque la nuova richiesta tra le richieste attive. Questo conferma che lo stato OVERLOAD non blocca l'acquisizione delle richieste.

7. Guasto tecnico

```
scenario GuastoAscensore
load ascensore.asm

begin generaGuasto
  set eventoGuasto := true;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := false;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := false;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step

  check statoErrore = GUASTO;
  check statoCabina = BLOCCATA;
  check statoPorte = CHIUSE;
  check direzione = NESSUNA;
  check timer = 10;
end
```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_aggiornaPersone()</i>	<i>Covered</i>	25.00%	25.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	33.33%	50.00%	27.27%	-
<i>r_main()</i>	<i>Covered</i>	50.00%	85.71%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	50.00%	50.00%	0.00%	33.33%
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-

Descrizione dello scenario

Lo scenario simula l'attivazione di un evento di guasto tecnico tramite `eventoGuasto = true`, in assenza di richieste e di variazioni nel numero di persone presenti in cabina.

Obiettivo

Verificare che il sistema entri correttamente nello stato GUASTO quando viene generato un evento di guasto.

Risultato

Quando viene attivato l'evento di guasto, il sistema passa nello stato GUASTO. La cabina viene bloccata, le porte vengono chiuse, la direzione viene annullata e il timer di ripristino viene inizializzato a 10.

8. Blocco delle richieste durante guasto

```
scenario BloccoRichiesteDuranteGuasto
load ascensore.asm

begin generaGuasto
  set eventoGuasto := true;
  set personeEntrate := 0;
  set personeUscite := 0;

  set pulsanteInterno(-1) := false;
  set pulsanteInterno(0) := false;
  set pulsanteInterno(1) := false;
  set pulsanteInterno(2) := false;
  set pulsanteInterno(3) := false;
  set pulsanteInterno(4) := false;

  set chiamataPianoSu(-1) := false;
  set chiamataPianoSu(0) := false;
  set chiamataPianoSu(1) := false;
  set chiamataPianoSu(2) := false;
  set chiamataPianoSu(3) := false;
  set chiamataPianoSu(4) := false;

  set chiamataPianoGiu(-1) := false;
  set chiamataPianoGiu(0) := false;
  set chiamataPianoGiu(1) := false;
  set chiamataPianoGiu(2) := false;
  set chiamataPianoGiu(3) := false;
  set chiamataPianoGiu(4) := false;

  step
  check statoErrore = GUASTO;
  check timer = 10;
end
```

```
begin richiestaDuranteGuasto
  set eventoGuasto := false;

  set pulsanteInterno(4) := true;

  step

  check statoErrore = GUASTO;
  check timer = 9;
  check richiesteAttive(4) = false;
  check statoCabina = BLOCCATA;
  check statoPorte = CHIUSE;
end
```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_aggiornaPersone()</i>	<i>Covered</i>	25.00%	25.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	66.67%	62.50%	36.36%	-
<i>r_main()</i>	<i>Covered</i>	75.00%	85.71%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	50.00%	50.00%	0.00%	33.33%
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-

Descrizione dello scenario

Lo scenario genera inizialmente una condizione di guasto tecnico, portando il sistema nello stato GUASTO. Successivamente viene simulata la pressione del pulsante interno del piano 4, verificando che la richiesta non venga acquisita.

Obiettivo

Verificare che, durante lo stato GUASTO, il sistema non acquisisca nuove richieste.

Risultato

Dopo l'ingresso nello stato di guasto viene simulata la pressione del pulsante interno del piano 4. Il modello non registra la richiesta e mantiene `richiesteAttive(4) = false`. Il sistema rimane nello stato GUASTO, con cabina bloccata, porte chiuse e timer in decremento.

9. Conservazione delle richieste durante guasto

```
scenario ConservazioneRichiesteGuasto
load ascensore.asm

begin acquisisciRichiestaPrimaDelGuasto
    set eventoGuasto := false;
    set personeEntrate := 0;
    set personeUscite := 0;

    set pulsanteInterno(-1) := false;
    set pulsanteInterno(0) := false;
    set pulsanteInterno(1) := false;
    set pulsanteInterno(2) := false;
    set pulsanteInterno(3) := true;
    set pulsanteInterno(4) := false;

    set chiamataPianoSu(-1) := false;
    set chiamataPianoSu(0) := false;
    set chiamataPianoSu(1) := false;
    set chiamataPianoSu(2) := false;
    set chiamataPianoSu(3) := false;
    set chiamataPianoSu(4) := false;

    set chiamataPianoGiu(-1) := false;
    set chiamataPianoGiu(0) := false;
    set chiamataPianoGiu(1) := false;
    set chiamataPianoGiu(2) := false;
    set chiamataPianoGiu(3) := false;
    set chiamataPianoGiu(4) := false;

    step

    check richiesteAttive(3) = true;
    check pianoCorrente = 1;
end
```

```
begin guastoConRichiestaGiaAttiva
    set pulsanteInterno(3) := false;
    set eventoGuasto := true;

    step

    check statoErrore = GUASTO;
    check statoCabina = BLOCCATA;
    check statoPorte = CHIUSE;
    check direzione = NESSUNA;
    check richiesteAttive(3) = true;
end
```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciAscensore()</i>	<i>Covered</i>	50.00%	70.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	25.00%	25.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	50.00%	50.00%	36.36%	-
<i>r_scegliDirezione()</i>	<i>Covered</i>	22.22%	26.32%	10.00%	-
<i>r_main()</i>	<i>Covered</i>	75.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	100.00%	100.00%	100.00%	66.67%
<i>r_muoviAscensore()</i>	<i>Covered</i>	33.33%	41.67%	33.33%	-
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-

Descrizione dello scenario

Lo scenario acquisisce inizialmente una richiesta interna verso il piano 3. Successivamente viene attivato un evento di guasto, verificando che la richiesta già registrata non venga cancellata dal sistema.

Obiettivo

Verificare che una richiesta già acquisita prima dell'ingresso nello stato GUASTO non venga cancellata.

Risultato

La richiesta del piano 3 viene acquisita prima dell'attivazione del guasto. Dopo l'ingresso nello stato GUASTO, la cabina viene bloccata, le porte vengono chiuse e la direzione viene annullata, ma la richiesta del piano 3 rimane attiva. Il modello conserva quindi correttamente le richieste già acquisite prima del guasto.

10. Ripristino dopo guasto

```

scenario RipristinoGuasto
load ascensore.asm

begin generaGuasto
    set eventoGuasto := true;
    set personeEntrate := 0;
    set personeUscite := 0;

    set pulsanteInterno(-1) := false;
    set pulsanteInterno(0) := false;
    set pulsanteInterno(1) := false;
    set pulsanteInterno(2) := false;
    set pulsanteInterno(3) := false;
    set pulsanteInterno(4) := false;

    set chiamataPianoSu(-1) := false;
    set chiamataPianoSu(0) := false;
    set chiamataPianoSu(1) := false;
    set chiamataPianoSu(2) := false;
    set chiamataPianoSu(3) := false;
    set chiamataPianoSu(4) := false;

    set chiamataPianoGiu(-1) := false;
    set chiamataPianoGiu(0) := false;
    set chiamataPianoGiu(1) := false;
    set chiamataPianoGiu(2) := false;
    set chiamataPianoGiu(3) := false;
    set chiamataPianoGiu(4) := false;

    step
        check statoErrore = GUASTO;
        check timer = 10;
end

```

```

begin decrementaTimer
    set eventoGuasto := false;

    step
        check timer = 9;
        check statoErrore = GUASTO;

    step
        check timer = 8;
        check statoErrore = GUASTO;

    step
        check timer = 7;
        check statoErrore = GUASTO;

    step
        check timer = 6;
        check statoErrore = GUASTO;

    step
        check timer = 5;
        check statoErrore = GUASTO;

    step
        check timer = 4;
        check statoErrore = GUASTO;

    step
        check timer = 3;
        check statoErrore = GUASTO;

    step
        check timer = 2;
        check statoErrore = GUASTO;

    step
        check timer = 1;
        check statoErrore = GUASTO;
end

```

```

    step
        check timer = 0;
        check statoErrore = GUASTO;
    end

begin ripristinaSistema
    step
        check timer = 0;
        check statoErrore = NESSUNO;
        check statoCabina = FERMA;
        check statoPorte = CHIUSE;
        check direzione = NESSUNA;
    end
end

```

<i>Regola</i>	<i>Stato</i>	<i>Branch coverage</i>	<i>Rule coverage</i>	<i>Update rule coverage</i>	<i>Forall rule coverage</i>
<i>r_gestisciAscensore()</i>	<i>Covered</i>	50.00%	50.00%	-	-
<i>r_aggiornaPersone()</i>	<i>Covered</i>	25.00%	25.00%	0.00%	-
<i>r_idle()</i>	<i>Covered</i>	-	100.00%	0.00%	-
<i>r_gestisciGuasto()</i>	<i>Covered</i>	83.33%	100.00%	54.55%	-
<i>r_main()</i>	<i>Covered</i>	100.00%	100.00%	-	-
<i>r_acquisisciRichieste()</i>	<i>Covered</i>	50.00%	50.00%	0.00%	33.33%
<i>r_gestisciErrori()</i>	<i>Covered</i>	50.00%	66.67%	-	-

Descrizione dello scenario

Lo scenario genera inizialmente un guasto tecnico, portando il sistema nello stato GUASTO con timer di ripristino inizializzato a 10. Nei passi successivi viene verificato il decremento progressivo del timer fino al completamento del ripristino.

Obiettivo

Verificare che, dopo l'ingresso nello stato GUASTO, il sistema decrementi correttamente il timer di ripristino e torni operativo quando il timer raggiunge 0.

Risultato

Dopo l'attivazione del guasto, il sistema entra nello stato GUASTO, blocca la cabina, chiude le porte, annulla la direzione e inizializza il timer a 10. Nei passi successivi il timer viene decrementato progressivamente fino a 0. Al passo logico successivo, il sistema torna operativo con `statoErrore = NESSUNO`, cabina FERMA, porte CHIUSE e direzione NESSUNA.

Generazione automatica di scenari con ATGT

Oltre agli scenari AVALLA definiti manualmente, è stato utilizzato ATGT per generare automaticamente ulteriori scenari di test.

Gli scenari generati automaticamente confermano la coerenza generale del modello anche su sequenze di input non costruite manualmente.

Alcuni scenari generati da ATGT contengono valori molto elevati o negativi per le variabili *personeEntrate* e *personeUscite*. Questo dipende dal fatto che tali variabili sono modellate come *Integer* e quindi ATGT può generare valori interi arbitrari.

Questi casi non rappresentano situazioni realistiche dal punto di vista applicativo, ma sono utili per osservare il comportamento logico del modello rispetto a configurazioni estreme.

Model checking con AsmetaSMV

Oltre alla validazione tramite scenari AVALLA manuali e scenari generati automaticamente con ATGT, il modello è stato analizzato tramite model checking con AsmetaSMV.

Il model checking è stato utilizzato per verificare proprietà CTL relative alla sicurezza del sistema e alla raggiungibilità di alcuni stati significativi.

A causa dell'elevato numero di stati generato dal modello completo, è stata realizzata una versione ridotta del modello ASM.

La versione ridotta mantiene la logica essenziale del sistema:

- acquisizione delle richieste;
- movimento della cabina;
- apertura e chiusura delle porte;
- gestione dello stato di guasto.

Rispetto al modello completo, sono stati limitati il numero di piani e il dominio del timer.

```
domain Piano = {0:2}
domain Timer = {0:2}

function tMax = 2
```

Inoltre, sono state rimosse alcune estensioni, come la gestione del sovraccarico e del numero di persone, per contenere lo spazio degli stati generato dal model checker.

Questa riduzione permette di verificare le proprietà principali evitando l'esplosione dello spazio degli stati.

Proprietà di sicurezza

Le principali proprietà di sicurezza verificate tramite AsmetaSMV riguardano il movimento della cabina, lo stato delle porte e la gestione del guasto.

In particolare, è stato verificato che:

- **P1:** la cabina non possa mai muoversi con le porte aperte;
- **P2:** la cabina possa muoversi solo in assenza di guasto;
- **P3:** durante un guasto la cabina venga bloccata;
- **P4:** durante un guasto le porte rimangano chiuse;
- **P5:** durante un guasto la direzione venga annullata;
- **P6:** se le porte sono aperte e il sistema non è in guasto, la cabina sia ferma;
- **P7:** se la cabina è bloccata, allora il sistema si trovi nello stato di guasto;
- **P8:** se il timer è maggiore di zero, allora il sistema sia ancora nello stato di guasto;
- **P9:** se il sistema è in guasto e il timer raggiunge zero, al passo successivo il sistema torni operativo.

```
// P1 - La cabina non deve mai muoversi con le porte aperte.
// Se la cabina è in movimento, allora le porte devono essere chiuse.
CTLSPEC ag(statoCabina = IN_MOVIMENTO implies statoPorte = CHIUSE)

// P2 - La cabina può muoversi solo se il sistema non è in stato di guasto.
// Questo impedisce movimenti durante condizioni anomale.
CTLSPEC ag(statoCabina = IN_MOVIMENTO implies statoErrore = NESSUNO)

// P3 - Durante il guasto la cabina deve essere bloccata.
// Il sistema non deve permettere movimento quando statoErrore = GUASTO.
CTLSPEC ag(statoErrore = GUASTO implies statoCabina = BLOCCATA)

// P4 - Durante il guasto le porte devono restare chiuse.
// Questa proprietà rappresenta una condizione di sicurezza.
CTLSPEC ag(statoErrore = GUASTO implies statoPorte = CHIUSE)

// P5 - Durante il guasto non deve esserci una direzione attiva.
// La direzione viene annullata impostandola a NESSUNA.
CTLSPEC ag(statoErrore = GUASTO implies direzione = NESSUNA)

// P6 - Se le porte sono aperte e il sistema non è in guasto,
// allora la cabina deve essere ferma.
CTLSPEC ag(statoPorte = APERTE and statoErrore = NESSUNO implies statoCabina = FERMA)

// P7 - Se la cabina è bloccata, allora il sistema deve essere in guasto.
// Nel modello ridotto l'unico stato anomalo modellato è GUASTO.
CTLSPEC ag(statoCabina = BLOCCATA implies statoErrore = GUASTO)

// P8 - Se il timer è maggiore di zero, allora il sistema deve essere in guasto.
// Il timer viene usato solo per gestire il ripristino dal guasto.
CTLSPEC ag(timer > 0 implies statoErrore = GUASTO)

// P9 - Se il sistema è in guasto con timer uguale a zero,
// al passo successivo deve tornare operativo.
CTLSPEC ag(statoErrore = GUASTO and timer = 0 implies ax(statoErrore = NESSUNO))
```

Proprietà di raggiungibilità

Oltre alle proprietà di sicurezza, sono state verificate anche proprietà di raggiungibilità.

In particolare, il model checking conferma che:

- **P10:** da ogni stato è possibile raggiungere uno stato senza guasto;
- **P11:** da ogni stato è possibile raggiungere uno stato con cabina ferma;
- **P12:** è raggiungibile un movimento verso l'alto;
- **P13:** è raggiungibile un movimento verso il basso;
- **P14:** è raggiungibile lo stato di guasto.

Queste proprietà permettono di verificare che il modello non rimanga bloccato in stati anomali non recuperabili e che i principali comportamenti del sistema siano effettivamente esplorabili.

```
// P10 - Da ogni stato è possibile raggiungere uno stato senza guasto.  
// Verifica che il guasto non sia permanente.  
CTLSPEC ag(ef(statoErrore = NESSUNO))  
  
// P11 - Da ogni stato è possibile raggiungere uno stato con cabina ferma.  
// Verifica che il sistema possa sempre tornare a uno stato stabile.  
CTLSPEC ag(ef(statoCabina = FERMA))  
  
// P12 - Esiste almeno un'esecuzione in cui la cabina può muoversi verso l'alto.  
// Verifica che il movimento verso piani superiori sia raggiungibile.  
CTLSPEC ef(direzione = SU and statoCabina = IN_MOVIMENTO)  
  
// P13 - Esiste almeno un'esecuzione in cui la cabina può muoversi verso il basso.  
// Verifica che il movimento verso piani inferiori sia raggiungibile.  
CTLSPEC ef(direzione = GIU and statoCabina = IN_MOVIMENTO)  
  
// P14 - Esiste almeno un'esecuzione in cui il sistema entra in guasto.  
// Verifica che lo stato GUASTO sia effettivamente raggiungibile.  
CTLSPEC ef(statoErrore = GUASTO)
```

Esito del model checking

L'esecuzione del model checking sul modello ridotto termina correttamente.

Le proprietà CTL definite risultano soddisfatte e confermano la coerenza del comportamento del modello rispetto agli aspetti di sicurezza considerati.

In particolare, il sistema impedisce il movimento della cabina con porte aperte, impedisce il movimento durante un guasto e garantisce che, in caso di guasto, la cabina venga bloccata, le porte vengano chiuse e la direzione venga annullata.

Il model checking conferma, inoltre, che gli stati principali del sistema sono raggiungibili e che il guasto non rappresenta uno stato permanente non recuperabile.

Conclusione

La validazione tramite scenari AVALLA manuali, scenari generati automaticamente con ATGT e model checking con AsmetaSMV conferma la coerenza del modello rispetto alle principali situazioni operative considerate e alle proprietà di sicurezza definite.

Gli scenari AVALLA manuali hanno permesso di verificare in modo mirato il funzionamento ordinario dell'ascensore e la gestione delle condizioni anomale, distinguendo correttamente tra sovraccarico e guasto.

Gli scenari generati con ATGT hanno permesso di esplorare automaticamente ulteriori configurazioni del modello, aumentando la confidenza nella correttezza del comportamento complessivo.

Il model checking con AsmetaSMV ha permesso di verificare formalmente proprietà CTL relative alla sicurezza del movimento, alla gestione delle porte, allo stato di guasto e alla raggiungibilità degli stati principali.

Nel complesso, il modello risulta coerente con i requisiti funzionali e con le principali proprietà di sicurezza definite nella documentazione di progetto.